Problem Set 2: ¿Cuál vivienda tiene piscina?

Big Data y Machine Learning para Economía Aplicada — 60-121 (NRC 10-993)

Universidad Icesi — Departamento de Economía
Prof. Eduard Fernando Martínez González

Información general

- **Publicación:** jueves 30 de octubre de 2026.
- **Fecha de entrega:** martes 17 de noviembre de 2026, a las 11:59 p. m. Subir al campus virtual un `.zip` con el `.pdf` del reporte, el código (`00_run.R` y scripts auxiliares) y el archivo `predicciones.csv` que produce `00_run.R`.
- **Peso:** 15 % de la nota final. Trabajo en parejas.
- **Datos:** dos bases y un diccionario (`diccionario_ps2.csv`), publicados en la carpeta `problem-sets/pset-2/` del repositorio del curso. La base `vivienda_entrenamiento.rds` (12.248 avisos) trae la variable `piscina`; la base `vivienda_prediccion.rds` (3.029 avisos) trae los mismos predictores **sin** `piscina`: ustedes nunca observan esa etiqueta.
- **Verificación:** en la plataforma del curso pueden subir su `predicciones.csv` cuantas veces quieran y recibir la matriz de confusión y sus métricas, calculadas sobre una mitad de la base de predicción escogida al azar (la plataforma no dice cuál).
- **Cómo se califica:** tres partes aplicadas que siguen la sesión 5, el ranking del curso y la reproducibilidad del código (rúbrica al final). No hay preguntas teóricas. Debrief con el ranking el 24 de noviembre.
- **IA:** permitida, declarando qué se usó y para qué.

La pregunta y las reglas del juego

Un portal inmobiliario quiere completar las fichas de sus avisos: saber, por ejemplo, qué viviendas tienen piscina, aunque el anuncio no venga con ese campo diligenciado. La y de este problem set se construyó leyendo el título y la descripción de los mismos avisos de Cali del Problem Set 1: `piscina` vale 1 si el texto la anuncia (el 48 % del entrenamiento). Ustedes no reciben el texto: la tarea es predecir `piscina` usando sólo lo estructurado del aviso — tipo, área, habitaciones, baños, ubicación, año y precio. Es una pregunta de **clasificación**, y el kit de esta entrega es el de la sesión 5: un árbol, y los ensambles que lo mejoran.

Cuatro reglas que valen para todo el problem set:

1. **La validación respeta los grupos.** Los avisos del mismo punto del mapa (misma `lat` y `lon`: el mismo aviso repetido, o varios apartamentos del mismo edificio) se parecen demasiado: si caen unos en el pliegue de ajuste y otros en el de validación, la validación se infla. La variable `grupo` de la base de entrenamiento identifica esos puntos: los pliegues se construyen de modo que **ningún grupo quede partido** (`groupKFold` de `caret`, o a mano sorteando grupos). Y no es un capricho: **la base de predicción no comparte ningún punto del mapa con la de entrenamiento**, así que copiar al vecino — el truco de los gemelos del Problem Set 1 — aquí no funciona, y una validación que reparta los grupos al azar les va a prometer una exactitud que la predicción no va a cumplir.
2. **Toda comparación va en la tabla del Anexo A:** misma validación y mismas métricas para todos los modelos — exactitud, sensibilidad, especificidad, precisión y F1, con la matriz de

confusión del modelo final.

3. **El ranking del curso.** `00_run.R` debe terminar escribiendo `predicciones.csv` con una fila por aviso de la base de predicción y dos columnas: `id` y `piscina_predicha` (0 o 1, sin probabilidades y sin `NA`). La plataforma les muestra sus métricas sobre una mitad de la base; **la nota del ranking se calcula con la otra mitad**, que la plataforma no conoce, así que optimizar contra la plataforma no reemplaza una validación honesta.

4. **Sólo las dos bases publicadas.** `00_run.R` puede leer únicamente las dos bases de la carpeta del problem set, `vivienda_entrenamiento` y `vivienda_prediccion`. Intentar recuperar etiquetas de la plataforma, o usar cualquier dato externo, descalifica del ranking.

1. Parte 1 — El árbol (25 pts)

Contexto. Antes de los ensambles, el modelo que se deja leer. Un árbol parte los avisos en regiones y predice la clase mayoritaria de cada hoja: verlo dibujado es ver *cómo* clasifica. Y antes del árbol, la referencia de toda clasificación: la clase mayoritaria.

Pregunta 1.

(a) Cargue las dos bases y describa lo esencial: número de avisos, prevalencia de `piscina` en el entrenamiento (total y por `tipo`), y un mapa sencillo (`plot` de `lon` contra `lat`) coloreado por `piscina`. ¿Qué variables tienen faltantes y qué va a hacer con ellas? Reporte la referencia de esta entrega: la exactitud del clasificador que asigna a todos los avisos la clase mayoritaria (fila 0 del Anexo A).

(b) Construya su esquema de validación: pliegues que no partan ningún `grupo` (regla 1). Explique en tres líneas qué pasaría si validara repartiendo avisos al azar, sabiendo que la base de predicción no comparte puntos del mapa con el entrenamiento.

(c) Estime un árbol de clasificación con `rpart` y sus opciones por defecto. Dibújelo y léalo: ¿cuál es el primer corte?, ¿qué pregunta hace cada rama?, y escoja dos hojas y describa qué avisos caen en ellas y con qué proporción de piscina. En tres líneas: ¿cómo clasifica este árbol?

(d) Afine la complejidad del árbol (la profundidad con `caret::train(method = 'rpart2')`, o el parámetro `cp` con la tabla de poda) usando su validación por grupos, y compare con el árbol por defecto en la tabla. Estime además el árbol *sin frenos* (`cp = 0`, `minsplit = 2`, `minbucket = 1`) y reporte su exactitud en el entrenamiento y en su validación: ¿qué le pasa y cómo se llama?

2. Parte 2 — Los ensambles: promediar y corregir (30 pts)

Contexto. El árbol es legible pero varía mucho: cambiar unos avisos cambia el árbol. Los ensambles de la sesión 5 atacan esa varianza por dos caminos: *bagging* y bosques aleatorios promedian cientos de árboles, y el *boosting* suma árboles pequeños que corrigen en fila lo que falta. Además del árbol de la Parte 1, deben presentar **al menos dos modelos más** de los vistos en clase; los dos siguientes son el mínimo.

Pregunta 2.

(a) Estime un *bagging* (un bosque con `mtry` igual al número de predictores) y un bosque aleatorio, afinando `mtry` con el error fuera de la bolsa (OOB) o con su validación por grupos. Compare las dos varas para el bosque que eligió: ¿el OOB y la validación por grupos cuentan lo mismo?, ¿cuál es más optimista aquí y por qué? (Pista: ¿el OOB respeta los grupos?)

(b) Estime un *boosting* (`xgboost`): afine el número de rondas con parada temprana usando **pliegues que respeten los grupos** (el argumento `folds` de `xgb.cv` recibe la lista de pliegues que

usted construya), y explore al menos otra perilla (la tasa de aprendizaje `eta` o la profundidad `max_depth`). Reporte la combinación elegida y su curva de validación.

(c) Para su mejor ensamble, reporte la importancia de variables (`varImpPlot` o `xgb.importance`) y léala en dos líneas: ¿qué variables mueven la predicción de piscina y tiene eso sentido económico? ¿Esa importancia dice que esas variables *causan* la piscina?

(d) Complete la tabla del Anexo A con todos los modelos y cierre con dos líneas: ¿cuánto le ganan los ensambles al árbol, y el mejor modelo a la logística de su equipo si decidió estimarla?

3. Parte 3 — El modelo final y la plataforma (15 pts)

Contexto. El cierre del *pipeline*: elegir con la validación, reajustar con todo el entrenamiento y predecir la base de predicción una sola vez. Como las clases están casi balanceadas, el umbral natural es 0,5 — pero la decisión es suya y se justifica.

Pregunta 3.

(a) Elija el modelo con mejor exactitud de validación, reajústelo con todo el entrenamiento y prediga `piscina` en la base de predicción. Si su modelo produce probabilidades, diga con qué umbral las convierte en 0/1 y por qué. Escriba en la primera línea del reporte: "Modelo final: ... Exactitud de validación: ...". Verifique que `predicciones.csv` tiene exactamente 3.029 filas y ninguna `NA`.

(b) Suba su archivo a la plataforma y reporte lo que devuelve (matriz y métricas). Compárelo con su validación por grupos: ¿coinciden? Si no, ¿qué lo explica?

(c) Cierre con un párrafo para el portal: qué tan bien se puede completar el atributo piscina con lo estructurado del aviso, en qué segmentos falla más el modelo (tipo, zona, precio), y qué *no* dice: por qué ni la importancia ni los cortes del árbol son efectos causales.

Rúbrica (100 puntos)

- (70 pts) Partes 1 a 3, con el peso indicado en cada título: procedimiento correcto, tablas y figuras pedidas, respuestas breves en lenguaje económico.
- (20 pts) Ranking del curso por la exactitud en la mitad de la base de predicción que la plataforma no conoce, calculada por el profesor con `predicciones.csv`: el primer equipo recibe 20 puntos, el último 10 y los demás en proporción a su posición. Un archivo que no se reproduce corriendo `00_run.R`, con filas faltantes, valores distintos de 0/1 o `NA`, ubica al equipo en el último lugar.
- (10 pts) Reproducibilidad y reporte: `00_run.R` corre de principio a fin sin intervención manual (rutas relativas a la carpeta del equipo, semilla fija, `sessionInfo()` al final) leyendo sólo las dos bases publicadas; extensión razonable, tablas con unidades, figuras legibles.

Anexo A: la tabla de resultados

Misma validación por grupos y mismas métricas para todos los modelos. La fila 0 se llena en la Parte 1(a), las del árbol en la Parte 1 y las de los ensambles en la Parte 2; la fila final, en la Parte 3. Agregue una fila por cada modelo adicional que decida presentar.

Modelo	Exactitud	Sensibilidad	Especificidad	Precisión	F1
M0: clase mayoritaria					
Árbol por defecto					
Árbol afinado					
Árbol sin frenos					
Bagging					
Bosque aleatorio					
Boosting					
Modelo final del Problem Set 2: ...					